

Spec — Gespräche Voice Input + Provider-Agnostic Review

mini-moi · Guild Created: 2026-06-14 — Claude.ai Updated: 2026-06-15 — Grok + Robert design input incorporated Status: spec_ready For: Claude Code Note: New feature — not a regression. Voice input exists on Wörter (Web Speech API, circular mic on the drill card, Chrome only) but was never built for Gespräche. This spec adds it, and extends the review layer to support per-session LLM selection.

Architecture: two independent layers

Layer	What it does	Phase	Default
Transcription	Audio → text (MediaRecorder → Whisper)	Phase 1	OpenAI Whisper
Review / LLM	Transcript analysis, feedback, Anki cards	Phase 1	Grok (xAI)

These are intentionally decoupled. Switching the review model mid-session doesn't require re-recording. Switching the transcription engine later (Phase 2) doesn't require touching the review layer. Each layer reads its provider from a request parameter, with a config-level default.

Priority: LLM / Review flexibility is higher priority than transcription flexibility. The model that generates corrections and Anki cards is what Robert is A/B testing. Transcription quality (Whisper) is a stable baseline for now.

UI layout (full session flow)

Persona: [Maria – Café Waitress ▼]

Scene: [Asking for the bill ▼]

Review: [Grok ▼] ← per-session selector, shown before recording

[● Record] [■ Stop] 00:00 Recording...

Transcript

(transcribed text populates here)

[Submit for Review]

Selector placement: above the Record button, not at the review step. Robert should commit to a model before speaking — A/B testing only works if the choice is deliberate and pre-committed. When Phase 2 adds transcription provider switching, a second selector row appears here (same location, not split across workflow steps).

Default: Grok. Consistent with the xAI-primary pattern across Curator and the existing German review pipeline.

Options in Phase 1: Grok | OpenAI | Gemini. Rendered as a simple dropdown or three segmented buttons — Claude Code to pick whichever is cleaner given the existing tab design.

Part 1 — Backend: `/api/transcribe` (unchanged from original)

```
python
```

```

@app.route('/app/german/api/transcribe', methods=['POST'])
@login_required
def transcribe_audio():
    if 'audio' not in request.files:
        return jsonify({'error': 'No audio file'}), 400

    audio_file = request.files['audio']
    provider = current_app.config.get('TRANSCRIBE_PROVIDER', 'openai')

    if provider == 'openai':
        transcript = transcribe_openai_whisper(audio_file)
    else:
        return jsonify({'error': f'Unknown transcription provider: {provider}'}), 400

    return jsonify({'transcript': transcript})

def transcribe_openai_whisper(audio_file):
    import openai
    client = openai.OpenAI(api_key=current_app.config['OPENAI_API_KEY'])
    audio_file.stream.seek(0)
    response = client.audio.transcriptions.create(
        model="whisper-1",
        file=("audio.webm", audio_file.stream, "audio/webm"),
        language="de",
        response_format="text"
    )
    return response

```

Config (`.env`):

```

TRANSCRIBE_PROVIDER=openai
OPENAI_API_KEY=sk-...

```

Part 2 — Backend: `/api/review` — provider-agnostic (updated)

The review endpoint accepts a `model` parameter and delegates to a dedicated provider module. The route handler stays thin — all provider logic lives in `providers/review_router.py`.

providers/review_router.py (new file)

python

```
"""
```

Provider-agnostic review router for Gespräche transcript analysis.

Add new providers here – no changes to the route handler needed.

```
"""
```

```
class ProviderError(Exception):
```

```
    pass
```

```
def run_review(transcript, persona, scene, model):
```

```
    """
```

Routes transcript review to the correct provider.

All providers return the same output shape:

```
    { feedback: [...], anki_cards: [...], model: str }
```

Raises ProviderError on failure – never silent-fallback.

```
    """
```

```
    if model == 'grok':
```

```
        return _review_grok(transcript, persona, scene)
```

```
    elif model == 'openai':
```

```
        return _review_openai(transcript, persona, scene)
```

```
    elif model == 'gemini':
```

```
        return _review_gemini(transcript, persona, scene)
```

```
    else:
```

```
        raise ProviderError(f'Unknown model: {model}')
```

```
def _review_grok(transcript, persona, scene):
```

```
    # xAI API call – key from config, prompt from german review templates
```

```
    ...
```

```
    return {'feedback': [...], 'anki_cards': [...], 'model': 'grok'}
```

```
def _review_openai(transcript, persona, scene):
```

```
    # OpenAI API call
```

```
    ...
```

```
    return {'feedback': [...], 'anki_cards': [...], 'model': 'openai'}
```

```
def _review_gemini(transcript, persona, scene):
```

```
    # Gemini API call
```

```
    ...
```

```
    return {'feedback': [...], 'anki_cards': [...], 'model': 'gemini'}
```

Route handler (thin — imports from router)

```
python

from providers.review_router import run_review, ProviderError

@app.route('/app/german/api/review', methods=['POST'])
@login_required
def review_transcript():
    data = request.get_json()
    transcript = data.get('transcript', '')
    model      = data.get('model', 'grok')
    persona    = data.get('persona', '')
    scene      = data.get('scene', '')

    if not transcript:
        return jsonify({'error': 'No transcript provided'}), 400

    try:
        result = run_review(transcript, persona, scene, model)
        return jsonify(result)
    except ProviderError as e:
        return jsonify({
            'error': f'Review failed ({model}): {str(e)}',
            'model': model
        }), 502
```

Adding a new provider later: add one function to `review_router.py` and one `elif` branch in `run_review()`. No other files change.

Part 3 — Frontend: Gespräche mic + model selector

Model selector (new)

```
html
```

```
<div class="gesprache-model-selector">
  <label>Review model</label>
  <select id="gesprache-model">
    <option value="grok"    selected>Grok</option>
    <option value="openai">OpenAI</option>
    <option value="gemini">Gemini</option>
  </select>
</div>
```

Voice recording controls (new)

html

```
<div class="gesprache-voice-controls">
  <button id="gesprache-record-btn" class="btn-record">● Record</button>
  <button id="gesprache-stop-btn"   class="btn-stop" style="display:none">■ Sto
  <span id="gesprache-timer"        class="recording-timer">00:00</span>
  <span id="gesprache-status"       class="recording-status"></span>
</div>
<textarea id="gesprache-transcript" ...></textarea>
```

JavaScript

javascript

```
let mediaRecorder = null;
let audioChunks = [];
let recordingTimer = null;
let secondsElapsed = 0;

const recordBtn      = document.getElementById('gesprache-record-btn');
const stopBtn        = document.getElementById('gesprache-stop-btn');
const timerEl        = document.getElementById('gesprache-timer');
const transcriptEl    = document.getElementById('gesprache-transcript');
const statusEl       = document.getElementById('gesprache-status');
const modelSelector   = document.getElementById('gesprache-model');

// — Recording —————

recordBtn.addEventListener('click', async () => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    audioChunks = [];
    secondsElapsed = 0;

    mediaRecorder = new MediaRecorder(stream, { mimeType: 'audio/webm' });
    mediaRecorder.ondataavailable = e => audioChunks.push(e.data);

    mediaRecorder.onstop = async () => {
      clearInterval(recordingTimer);
      stream.getTracks().forEach(t => t.stop());
      stopBtn.style.display = 'none';
      recordBtn.disabled = false;
      await transcribeAudio();
    };

    mediaRecorder.start();
    recordBtn.disabled = true;
    stopBtn.style.display = 'inline-block';
    statusEl.textContent = 'Recording...';

    recordingTimer = setInterval(() => {
      secondsElapsed++;
      const mm = String(Math.floor(secondsElapsed / 60)).padStart(2, '0');
      const ss = String(secondsElapsed % 60).padStart(2, '0');
      timerEl.textContent = `${mm}:${ss}`;
    }, 1000);
```



```
    } catch (err) {
      statusEl.textContent = 'Microphone access denied.';
    }
  });
```

```
stopBtn.addEventListener('click', () => {
  if (mediaRecorder && mediaRecorder.state !== 'inactive') {
    mediaRecorder.stop();
  }
});
```

```
// — Transcription —
```

```
async function transcribeAudio() {
  statusEl.textContent = 'Transcribing...';
  const blob = new Blob(audioChunks, { type: 'audio/webm' });
  const formData = new FormData();
  formData.append('audio', blob, 'session.webm');

  try {
    const res = await fetch('/app/german/api/transcribe', {
      method: 'POST', body: formData
    });
    const data = await res.json();
    if (data.transcript) {
      transcriptEl.value = data.transcript;
      statusEl.textContent = 'Transcript ready – review and submit.';
    } else {
      statusEl.textContent = 'Transcription failed. Paste manually.';
    }
  } catch (err) {
    statusEl.textContent = 'Error connecting to transcription service.';
  }
}
```

```
// — Review submission —
```

```
async function submitReview(transcript, model, persona, scene, attempt = 1) {
  const MAX_ATTEMPTS = 2;
  const RETRY_DELAY_MS = 1500;

  try {
    const res = await fetch('/app/german/api/review', {
      method: 'POST',
```

```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ transcript, model, persona, scene })
  });
  const data = await res.json();

  if (data.error) {
    // Retry once on 502/503 (transient provider issues, rate limits)
    if (attempt < MAX_ATTEMPTS && res.status >= 500) {
      statusEl.textContent = `${model} failed - retrying...`;
      await new Promise(r => setTimeout(r, RETRY_DELAY_MS));
      return submitReview(transcript, model, persona, scene, attempt + 1);
    }
    // After retry or on 4xx - show clear error, name the model
    statusEl.textContent = `❌ ${data.error}`;
    return;
  }

  // Success - show which model actually ran (for A/B tracking)
  statusEl.textContent = `✓ Review from ${data.model} complete.`;
  renderReview(data);
} catch (err) {
  if (attempt < MAX_ATTEMPTS) {
    statusEl.textContent = `${model} unreachable - retrying...`;
    await new Promise(r => setTimeout(r, RETRY_DELAY_MS));
    return submitReview(transcript, model, persona, scene, attempt + 1);
  }
  statusEl.textContent = `Error submitting to ${model}. Try again or switch m
}
}

document.getElementById('gesprache-submit-btn').addEventListener('click', async
  const transcript = transcriptEl.value.trim();
  const model      = modelSelector.value;
  const persona    = document.getElementById('gesprache-persona').value;
  const scene      = document.getElementById('gesprache-scene').value;

  if (!transcript) {
    statusEl.textContent = 'No transcript to submit.';
    return;
  }

  statusEl.textContent = `Submitting to ${model}...`;

```

```
    await submitReview(transcript, model, persona, scene);
  });
```

Error UX note: when a model fails, the status message names the model that failed and suggests switching. Robert can see exactly what happened and make a deliberate choice — no silent auto-fallback.

Phasing

Phase	What ships	This spec?
1	MediaRecorder + Whisper transcription + per-session LLM selector (Grok/OpenAI/Gemini)	✓ Yes
2	Transcription provider selector (add Grok transcription as option)	Deferred
3	Deeper Grok Voice integration	Deferred

Definition of Done

Backend:

- ☐ `providers/review_router.py` created — `run_review()`, `ProviderError`, `_review_grok()`, `_review_openai()`, `_review_gemini()` all returning same output shape `{feedback, anki_cards, model}`
- ☐ `/api/transcribe` — Whisper, `language=de`, returns `{transcript}`
- ☐ `/api/review` — thin route handler, imports from `review_router`, accepts `model` param, never silent-fallback
- ☐ All three API keys in `.env` and config

Frontend:

- ☐ Model selector (Grok default) appears above Record button
- ☐ Record / Stop / timer work correctly
- ☐ Transcription populates textarea on stop
- ☐ Submit passes `model` to `/api/review`
- ☐ Error states name the failing model, suggest switching

- ☐ Response shows which model ran (`data.model`)

End-to-end test:

- ☐ Record short German sentence → Whisper transcript correct
 - ☐ Record full ~3 min session → transcript quality acceptable
 - ☐ Submit with Grok → review renders correctly
 - ☐ Switch to OpenAI → review renders correctly
 - ☐ Force a provider error → error message names the model, no crash
 - ☐ Robert confirms: record → transcribe → select model → submit → review without leaving the browser
-

Commit

```
bash
```

```
git add portal/routes/german.py portal/templates/german_gesprache.html \
      portal/static/german.js portal/providers/review_router.py
git commit -m "feat: Gespräche voice input + provider-agnostic review"
```

```
MediaRecorder + OpenAI Whisper transcription (/api/transcribe, language=de).
Per-session LLM selector (Grok default, OpenAI/Gemini options) shown before
recording. providers/review_router.py: clean provider abstraction with
run_review() router, ProviderError, _review_grok/openai/gemini() all
returning same output shape. /api/review stays thin – imports from router.
Frontend: one retry with 1.5s backoff on 5xx before surfacing error; error
messages name the failing model. A/B tracking via data.model in response.
OPENAI_API_KEY + GEMINI_API_KEY added to config."
git push origin main
```

Deferred

- **Phase 2 — Smart fallback:** if Grok fails after retry, automatically try OpenAI once before surfacing an error. Show Robert which model ultimately ran. (Not Phase 1 — Robert should commit to a model per session; fallback blurs that. Revisit after A/B testing establishes which models are actually unreliable.)
- **Phase 2 — Transcription provider selector** (Grok transcription option)
- **Phase 3 — Deeper Grok Voice integration**

- Profile-stored default model preference (add after A/B testing reveals a stable preference)
 - Turn-by-turn recording (record each speaker turn separately)
 - Speaker diarization (auto-label "Sie" vs persona turns)
 - Cost/token tracking per model (low priority logging addition)
 - Model capability matrix config (which models support what)
-

Spec · Gespräche Voice Input + Provider-Agnostic Review · 2026-06-14 Updated: 2026-06-15